

BANKING DATA ANALYSIS

SQL Queries, Data Quality Checks & Business Insights

DATA QUALITY ASSESSMENT

SQL ANALYSIS	SQL QUERY	OUTPUT	INSIGHT
Duplicate Record Check (Customers, Accounts, Transaction	<pre>SELECT Email, COUNT(*) AS Duplicate_Count FROM Customers GROUP BY Email HAVING COUNT(*) > 1; --REMOVAL OF DUPLICATES DELETE FROM Customers c WHERE c.Customer_ID NOT IN (SELECT MIN(Customer_ID) FROM Customers GROUP BY Email); SELECT Phone, COUNT(*) AS Duplicate_Count FROM Customers GROUP BY Phone HAVING COUNT(*) > 1; --REMOVAL OF DUPLICATES DELETE FROM Customers c WHERE c.Customer_ID NOT IN (SELECT MIN(Customer_ID) FROM Customers GROUP BY Phone); SELECT Customer_ID, Account_Type, COUNT(*) AS Duplicate_Count FROM Accounts GROUP BY Customer_ID, Account_Type HAVING COUNT(*) > 1; --REMOVAL OF DUPLICATES DELETE FROM Accounts WHERE Account_ID NOT IN (SELECT MIN(Account_ID) FROM Accounts GROUP BY Customer_ID, Account_Type); SELECT Account_ID, Transaction_Date, Amount, Transaction_Type, COUNT(*) AS Duplicate_Count FROM Bank_Transactions GROUP BY Account_ID, Transaction_Date, Amount, Transaction_Type HAVING COUNT(*) > 1; --REMOVAL OF DUPLICATES DELETE FROM Bank_Transactions WHERE Transaction_ID NOT IN (SELECT MIN (Transaction_ID) FROM Bank_Transactions GROUP BY Account_ID, Transaction_Date, Amount, Transaction_Type);</pre>	No duplicate records found.	Confirms the dataset is clean and free from duplicate entries, ensuring reliable analysis.

Null Value Check	SELECT * FROM Customers WHERE (Customer_Name IS NULL OR Gender IS NULL OR City IS NULL OR Phone IS NULL OR Email IS NULL);	No null values found.	
Data Validation Checks	SELECT * FROM Customers WHERE MONTHS_BETWEEN(SYSDATE, Date_of_Birth) / 12 < 18; SELECT * FROM Customers WHERE Monthly_Income <= 0; SELECT * FROM Accounts WHERE Balance < 0; SELECT * FROM Bank_Transactions WHERE Amount <= 0; SELECT * FROM Accounts A WHERE NOT EXISTS (SELECT 1 FROM Customers C WHERE C.Customer_ID = A.Customer_ID); SELECT * FROM Bank_Transactions T WHERE NOT EXISTS (SELECT 1 FROM Accounts A WHERE A.Account_ID = T.Account_ID);	No invalid ages, negative balances, invalid emails, invalid phone numbers, or orphan records found.	

DATABASE OVERVIEW (EDA)

SQL ANALYSIS	SQL QUERY	OUTPUT
Total Customers	SELECT COUNT(*) AS Total_Customers FROM Customers; SELECT COUNT(*) AS Total_Customers FROM Customers;	100
Total Branches	SELECT COUNT(*) AS Total_Branches FROM Branches;	10
Total Accounts	SELECT COUNT(*) AS Total_Accounts FROM Accounts;	150
Total Transactions	SELECT COUNT(*) AS Total_Transactions FROM Bank_Transactions	1000
Total Loans	SELECT COUNT(*) AS Total_Loans FROM Loans;	100
Total Deposits	SELECT SUM(Amount) AS Total_Deposits FROM Bank_Transactions WHERE Transaction_Type = 'Deposit';	250,500
Average Account Balance	SELECT AVG(Balance) AS Average_Balance FROM Accounts;	198,750
Highest And Lowest Transaction	SELECT MAX(Amount) AS Highest_Transaction FROM Bank_Transactions;	Highest = 1500, Lowest = 501

	SELECT MIN(Amount) AS Lowest_Transaction FROM Bank_Transactions;	
Total Bank Balance	SELECT SUM(Balance) AS Total_Bank_Balance FROM Accounts;	29,812,500

CUSTOMER ANALYSIS

SQL ANALYSIS	SQL QUERY	OUTPUT	INSIGHT														
Customer Distribution By City	SELECT City, COUNT(*) AS Customer_Count FROM Customers GROUP BY City ORDER BY Customer_Count DESC;	<table><tr><th>CITY</th><th>CUSTOMER_COUNT</th></tr><tr><td colspan="2">-----</td></tr><tr><td>Karachi</td><td>20</td></tr><tr><td>Islamabad</td><td>20</td></tr><tr><td>Lahore</td><td>20</td></tr><tr><td>Faisalabad</td><td>20</td></tr><tr><td>Multan</td><td>20</td></tr></table>	CITY	CUSTOMER_COUNT	-----		Karachi	20	Islamabad	20	Lahore	20	Faisalabad	20	Multan	20	
CITY	CUSTOMER_COUNT																

Karachi	20																
Islamabad	20																
Lahore	20																
Faisalabad	20																
Multan	20																
Customer Distribution By Gender	SELECT Gender, COUNT(*) AS Total_Customers FROM Customers GROUP BY Gender;	<table><tr><th>GENDER</th><th>TOTAL_CUSTOMERS</th></tr><tr><td colspan="2">-----</td></tr><tr><td>Female</td><td>50</td></tr><tr><td>Male</td><td>50</td></tr></table>	GENDER	TOTAL_CUSTOMERS	-----		Female	50	Male	50	S						
GENDER	TOTAL_CUSTOMERS																

Female	50																
Male	50																
Average Income By Occupation	SELECT Occupation, AVG(Monthly_Income) AS Average_Income FROM Customers GROUP BY Occupation ORDER BY Average_Income DESC;	Engineers have the highest average income (76,000).	Engineers appear to be the highest-earning customer segment in the dataset.														
Top 10 High Value Customers	WITH Customer_Value AS (SELECT C.Customer_ID, C.Customer_Name, SUM(A.Balance) AS Total_Balance, RANK() OVER(ORDER BY SUM(A.Balance) DESC) AS Rank_No FROM Customers C JOIN Accounts A ON C.Customer_ID=A.Customer_ID GROUP BY C.Customer_ID, C.Customer_Name) SELECT * FROM Customer_Value WHERE Rank_No <=10;	Top 10 customers ranked by total balance.	Identifies the bank's most valuable customers for relationship management.														
Customers With Multiple Accounts	SELECT Customer_ID, COUNT(Account_ID) AS Number_of_Accounts FROM Accounts GROUP BY Customer_ID HAVING COUNT(Account_ID) > 1;	First 50 customers have two 21 accounts each.	Half of the customers maintain multiple banking accounts.														
Customer Account Summary	SELECT C.Customer_Name, COUNT(A.Account_ID) AS Accounts, SUM(A.Balance) AS Total_Balance FROM Customers C JOIN Accounts A ON C.Customer_ID=A.Customer_ID GROUP BY C.Customer_Name ORDER BY Total_Balance DESC;	Customer_50 has the highest combined balance (520,000).	Summarizes each customer's total accounts and combined balances.														

ACCOUNT AND TRANSACTION ANALYSIS

SQL ANALYSIS	SQL QUERY	OUTPUT	INSIGHT
Account Type Distribution	SELECT Account_Type, COUNT(*) AS Total_Accounts FROM Accounts GROUP BY Account_Type;	Savings = 50, Current = 50, Business = 50	All account types are equally represented.

Balance By Account Type	SELECT Account_Type, SUM(Balance) AS Total_Balance FROM Accounts GROUP BY Account_Type ORDER BY Total_Balance DESC;	Savings accounts hold the highest balance.	Savings accounts contribute the largest share of total account balances.
Transaction Count By Type	SELECT Transaction_Type, COUNT(*) AS Number_of_Transactions FROM Bank_Transactions GROUP BY Transaction_Type;	250 transactions for each type.	Transaction activity is evenly distributed among deposits, withdrawals, payments, and transfers.
Transaction Amount By Channel	SELECT Channel, SUM(Amount) AS Total_Amount FROM Bank_Transactions GROUP BY Channel ORDER BY Total_Amount DESC;	Online has the highest total amount.	Online banking is the leading transaction channel by value.
Recent Transactions	SELECT * FROM Bank_Transactions ORDER BY Transaction_Date DESC FETCH FIRST 20 ROWS ONLY;	Latest 20 transactions displayed.	Useful for monitoring the most recent banking activities.
Top 10 Deposits	SELECT * FROM (SELECT * FROM BANK_TRANSACTIONS WHERE TRANSACTION_TYPE = 'Deposit' ORDER BY AMOUNT DESC) FETCH FIRST 10 ROWS ONLY;	Displays the 10 highest-value deposit transactions.	Identifies major deposit inflows and highlights high- value transactions for further analysis.
Account Activity Audit	-- CREATING LOG TABLE CREATE TABLE ACCOUNT_LOG (LOG_ID NUMBER GENERATED ALWAYS AS IDENTITY, ACCOUNT_ID NUMBER, ACTION_TYPE VARCHAR2(10), ACTION_USER VARCHAR2(100), ACTION_TIME TIMESTAMP); -- CREATING TRIGGER CREATE OR REPLACE TRIGGER LOG_ON_ACCOUNTS AFTER INSERT OR UPDATE OR DELETE ON ACCOUNTS FOR EACH ROW BEGIN IF INSERTING THEN INSERT INTO ACCOUNT_LOG (ACCOUNT_ID, ACTION_TYPE, ACTION_USER, ACTION_TIME) VALUES (:NEW.ACCOUNT_ID, 'INSERT', USER, SYSTIMESTAMP); ELSIF UPDATING THEN INSERT INTO ACCOUNT_LOG (ACCOUNT_ID, ACTION_TYPE, ACTION_USER, ACTION_TIME) VALUES (:NEW.ACCOUNT_ID, 'UPDATE', USER, SYSTIMESTAMP); ELSIF DELETING THEN INSERT INTO ACCOUNT_LOG (ACCOUNT_ID, ACCOUNT_TYPE, ACTION_USER, ACTION_TIME) VALUES (:OLD.ACCOUNT_ID, 'DELETE', USER, SYSTIMESTAMP); END IF; END; /	Records the user, action performed (INSERT/UPDATE/DELETE), affected account, and date/time of the action.	Provides an audit trail of changes made to the Accounts table, improving accountability, traceability, and data security.

Account Details Retrieval	<pre>CREATE OR REPLACE PROCEDURE get_account_details (p_account_id IN ACCOUNTS.ACCOUNT_ID%TYPE, p_account_type OUT ACCOUNTS.ACCOUNT_TYPE%TYPE, p_opening_date OUT ACCOUNTS.OPENING_DATE%TYPE, p_balance OUT ACCOUNTS.BALANCE%TYPE, p_status OUT ACCOUNTS.ACCOUNT_STATUS%TYPE) AS BEGIN SELECT ACCOUNT_TYPE, OPENING_DATE, BALANCE, ACCOUNT_STATUS INTO p_account_type, p_opening_date, p_balance, p_status FROM ACCOUNTS WHERE ACCOUNT_ID = p_account_id; END; / SET SERVEROUTPUT ON; DECLARE v_account_type ACCOUNTS.ACCOUNT_TYPE%TYPE; v_opening_date ACCOUNTS.OPENING_DATE%TYPE; v_balance ACCOUNTS.BALANCE%TYPE; v_status ACCOUNTS.ACCOUNT_STATUS%TYPE; BEGIN get_account_details(101, v_account_type, v_opening_date, v_balance, v_status); DBMS_OUTPUT.PUT_LINE('Account Type: ' v_account_type); DBMS_OUTPUT.PUT_LINE('Opening Date: ' v_opening_date); DBMS_OUTPUT.PUT_LINE('Balance: ' v_balance); DBMS_OUTPUT.PUT_LINE('Account Status: ' v_status); END; /</pre>	ACCOUNT_TYPEOPENING_DBALANCEACCOUNT_STATUSBusiness11-APR-20262500Active	This procedure provides a quick and efficient way to retrieve complete information about a specific account using its unique ACCOUNT_ID. It allows users or banking applications to access important account details without writing the SQL query repeatedly.
---------------------------	---	--	---

BRANCH ANALYSIS

SQL ANALYSIS	SQL QUERY	OUTPUT	INSIGHT
Branch Wise Account Analysis	<pre>CREATE OR REPLACE VIEW Branch_Performance AS SELECT b.branch_id, b.branch_name, COUNT(DISTINCT a.customer_id) AS total_customers, COUNT(a.account_id) AS total_accounts, NVL(SUM(a.balance),0) AS total_deposits FROM branches b LEFT JOIN accounts a ON b.branch_id = a.branch_id GROUP BY b.branch_id,</pre>	Hyderabad branch has the highest total balance.	Helps compare branch performance based on account.

	b.branch_name;		
Branch Ranking By Total Balance	SELECT B.Branch_Name, SUM(A.Balance) AS Branch_Balance, RANK() OVER(ORDER BY SUM(A.Balance) DESC) AS Branch_Rank FROM Branches B JOIN Accounts A ON B.Branch_ID=A.Branch_ID GROUP BY B.Branch_Name;	Hyderabad ranked 1st.	Ranks branches based on the total balances they manage.
Branch Ranking By Number Of Customers	SELECT B.Branch_Name, COUNT(DISTINCT A.Customer_ID) AS Customers, DENSE_RANK() OVER(ORDER BY COUNT(DISTINCT A.Customer_ID) DESC) Rank_No FROM Branches B JOIN Accounts A ON B.Branch_ID=A.Branch_ID GROUP BY B.Branch_Name;	All branches have 10 customers, is evenly distributed.	Indicates customer allocation across branches.

LOAN ANALYSIS

SQL ANALYSIS	SQL QUERY	OUTPUT	INSIGHT
Loan Amount By Loan Type	SELECT Loan_Type, SUM(Loan_Amount) AS Total_Loan_Amount FROM Loans GROUP BY Loan_Type ORDER BY Total_Loan_Amount DESC;	Home loans have the highest loan amount.	Home loans contribute the largest portion of the loan portfolio.
Loan Status Analysis	CREATE OR REPLACE VIEW Loan_Analysis AS SELECT loan_type, COUNT(*) AS total_loans, SUM(loan_amount) AS total_loan_amount, AVG(loan_amount) AS average_loan_amount, SUM(outstanding_amount) AS outstanding_balance FROM loans GROUP BY loan_type;	Pending = 34, Approved = 33, Closed = 33	Loan statuses are almost evenly distributed, with pending loans slightly higher.